How a Unified Development Toolchain Supports Automated Testing Consistency in a TypeScript Full-Stack Monorepo: A Case Study of Circles

Aldiyar Seylkhanov
Bachelor Degree Student, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
seylkhanov.aldiyar@gmail.com

Alexandr Tyulkov
Master Degree Student, School of Software Engineering
Astana IT University (AITU)
Astana, Kazakhstan
widesehl@gmail.com
0009-0009-6422-0559

***Abstract*—TypeScript monorepos often combine frontend, backend, and shared packages under one repository, but test execution is still shaped by package-local configuration. This paper studies how a unified toolchain affects testing consistency in such a setting. The Circles repository is used as a case study because it runs backend, frontend, and shared-package tests through the Vite+ command interface while still keeping a small number of package-specific overrides. The evaluation uses repository inspection, CI workflow analysis, and direct execution of test and coverage commands. The observed configuration surface consists of nine testing and CI configuration files, seven automated test files, and two GitHub Actions workflows. Empirical results show stable execution in the backend and shared package, with 21 backend tests and one shared-package test passing, while the aggregated frontend run exposes a configuration-level failure that does not appear when the frontend unit project is executed in isolation. This result is important because it shows that a unified command surface improves consistency, but multi-project composition can still preserve hidden environment differences. The paper therefore argues that unified toolchains reduce configuration fragmentation and improve reproducibility, yet consistency must still be evaluated at the combined pipeline level rather than only at the per-project level.**

***Index Terms*—monorepo, TypeScript, unified toolchain, automated testing, Vite, software engineering**

I. Introduction

TypeScript monorepos often combine frontend applications, backend services, and shared packages in one repository. This structure simplifies code sharing, but it also increases toolchain complexity. Different packages may use different test runners, module resolvers, transform steps, and command interfaces. As a result, the same test logic can behave differently across packages even when the application code is correct. In this setting, inconsistency comes from the execution environment rather than from the tests themselves.

This problem is important in full-stack repositories because packages are not isolated in practice. Shared types, utility modules, and common build assumptions move across package boundaries. When each package defines its own testing stack, small differences in resolution rules, TypeScript transforms, or runtime defaults can produce conflicting results. A test may pass in one package and fail in another because the surrounding toolchain is different. The main engineering issue is therefore configuration fragmentation.

This paper examines that issue through the Circles project, a TypeScript full-stack monorepo that contains a React frontend, a Hono backend, and shared packages. The study analyzes the current Vite+ based toolchain as an example of partial unification: packages share one command interface and one main testing stack, but some package-specific test configuration remains. The analysis combines repository inspection, CI workflow review, and direct execution of the available test and coverage commands. It is guided by the following research questions:

Research Question 1 (RQ1) How does per-package toolchain fragmentation create inconsistent test behavior across packages in a TypeScript monorepo?

Research Question 2 (RQ2) To what extent does a unified toolchain reduce the configuration surface required to run tests across the monorepo?

Research Question 3 (RQ3) Does sharing one CLI entry point, one resolver, and one transform pipeline produce a more uniform testing and CI execution model in Circles?

The paper makes three contributions. First, it defines toolchain fragmentation as an environment-level source of testing inconsistency in TypeScript monorepos. Second, it documents the current Circles testing architecture with observable repository evidence rather than hypothetical design claims. Third, it evaluates the effect of unification using configuration surface size, test environment uniformity, CI pipeline structure, and directly observed execution outcomes. The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the Circles monorepo and the unified toolchain. Section IV presents the evaluation. Section V discusses limitations and generalizability. Section VI concludes the paper.

II. Related Work

Prior work relevant to this study falls into two main areas: empirical studies of software quality and fault pat-

terns in JavaScript and TypeScript projects, and studies of automated testing in continuous integration environments. These papers explain why reliability problems persist in large TypeScript systems and why automation infrastructure matters, but they do not directly examine unified test environments inside a monorepo.

A. TypeScript Quality and Fault Profiles

Bogner and Merkel compared 604 JavaScript and TypeScript GitHub projects across code quality, understandability, bug proneness, and bug resolution time [1]. Their results suggest that TypeScript projects have better code quality and understandability, but they do not show significantly lower bug proneness or faster bug resolution. This is relevant because it limits a common explanation of quality improvement. TypeScript may improve some properties of code, yet language choice alone does not remove the broader conditions that produce bugs during development and testing.

Tang, Alimadadi, and Sumner move closer to the present paper by analyzing 633 bug reports from 16 TypeScript repositories and constructing a taxonomy of fault categories [2]. Their main finding is that tooling and configuration faults, API misuse, and asynchronous error handling are more prominent than pure logic or syntax mistakes. They further argue that many failures arise at integration and orchestration boundaries. This directly supports the premise of the current study. In mature TypeScript ecosystems, fragility is often located in the surrounding toolchain rather than in local program logic.

These two studies are important, but they stop short of the present research question. Bogner and Merkel compare language populations at repository scale, while Tang et al. characterize fault patterns across the TypeScript ecosystem. Neither paper examines whether using one shared resolver, one transform pipeline, and one CLI entry point across packages changes test consistency inside a single full-stack monorepo.

B. Continuous Integration and Automated Testing Infrastructure

Wang et al. study test automation maturity in CI contexts and report that higher maturity is associated with better product quality and shorter release cycles [3]. Their results show that reliable automated testing infrastructure matters to project outcomes. However, the unit of analysis is the CI practice of a project as a whole, not the consistency of test environments across packages within one repository.

Yu et al. examine automated non-functional requirement testing in CI environments through a multi-case industry study [4]. Their findings describe CI as an environment composed of tools, components, metrics, and feedback mechanisms that enable and support different forms of testing. This is useful for the present paper because it emphasizes that test behavior depends on infrastructure, not only on test cases. At the same time, the study is concerned with NFR evaluation in CI environments rather than with per-package differences in local test stacks.

The CI literature therefore establishes that automated testing quality depends on the surrounding environment and that shared infrastructure can improve evaluation and feedback. What it does not isolate is the narrower problem examined here: the same monorepo may still run different test runners, transforms, and resolution rules in different packages even when all tests are automated in CI.

In summary, the reviewed literature shows three things. First, TypeScript improves some code-level quality indicators but does not by itself eliminate bug-related problems [1]. Second, current TypeScript failures often cluster around configuration, integration, and tooling boundaries [2]. Third, CI research shows that testing outcomes depend heavily on automation infrastructure [3]; [4]. The remaining gap is the effect of toolchain unification on testing consistency across packages in a TypeScript monorepo. That gap motivates the Circles case study.

III. Methodology

A. Case Study Context

Circles is a full-stack TypeScript monorepo with three active package areas relevant to testing. The frontend package uses React and contains both browser-level component tests and Playwright end-to-end tests. The backend package exposes API and worker code and uses Vitest for logic-level tests. The shared packages/utils package contains reusable helpers and its own test file. Across the repository, the dominant command surface is Vite+, exposed through the vp CLI.

The current repository state provides a suitable midterm case because it combines a unified command interface with package-level specialization. The backend and shared package each define testing inside vite.config.ts. The frontend keeps one base Vite config plus dedicated Vitest project files for unit and browser execution and a separate Playwright config for end-to-end tests. This structure permits direct observation of where unification succeeds and where configuration divergence remains.

TABLE I
Observed package structure and testing stack in Circles

Package / Area	Primary role	Observed test stack	Observed test files
Frontend	UI and route rendering	Vitest unit project, Vitest browser project, Playwright	3
Backend	API, workers, helper logic	Vitest through apps/backend/vite.config.ts	4
Shared utils	Reusable helper package	Vitest through packages/utils/vite.config.ts	1

B. Data Collection Procedure

The evaluation used only repository-observable evidence. Five classes of data were collected.

First, the repository structure was inspected to identify configuration files, package scripts, and test locations. Second, the CI workflows in .github/workflows/ci.yml and .github/workflows/e2e.yml were reviewed to document the execution model used on push and pull request events. Third, direct test execution was performed with the same package commands defined in the repository: backend tests, frontend tests, frontend unit coverage, backend coverage, and shared-package coverage. Fourth, execution logs were examined for failures, warnings, and timing information. Fifth, coverage reports were used to identify the current automated scope and any detectability gaps.

The concrete commands executed during the evaluation were vp run @circles/backend#test, vp run @circles/frontend#test, vp run @circles/backend#test:coverage, vp run @circles/frontend#test:coverage, and vp run utils#test:coverage. All commands were run against the repository state observed on 2026-04-10.

C. Evaluation Dimensions

The analysis used four dimensions that match the midterm requirement to connect engineering choices with measurable evidence.

1) **Configuration surface size.** This dimension counts the number of files that directly define test or CI behavior.
2) **Execution consistency.** This dimension checks whether equivalent test commands behave uniformly across packages and across isolated versus aggregated runs.
3) **Coverage and detectability.** This dimension evaluates which modules have automated evidence and whether coverage reports indicate strong or weak visibility into failures.
4) **Pipeline reproducibility.** This dimension documents how repository commands are mapped into CI workflows.

TABLE II
Evaluation dimensions and evidence sources

Dimension	Metric	Repository evidence
Configuration surface	Count of config files	Vite, Vitest, Playwright, and GitHub Actions files
Execution consistency	Pass or fail outcome	Direct command outputs from backend, frontend, and shared package
Coverage and detectability	Coverage percentage	V8 coverage reports emitted by package coverage commands
Pipeline reproducibility	Step mapping	Workflow YAML files and package scripts

D. Configuration and Pipeline Model

Nine files were identified as direct parts of the testing and CI configuration surface: two GitHub Actions workflows, three vite.config.ts files, three frontend Vitest configuration files, and one Playwright configuration file. This count is small for a multi-package repository, but it is not zero. The result matters because it shows that unification in Circles is operational rather than absolute.

TABLE III
Observed CI pipeline structure across the two workflows

Push / PR	→	Install	→	Check and test	→	Build or E2E
GitHub event	→	pnpm install	→	pnpm vp check, pnpm vp run test -r	→	pnpm vp run build -r or Playwright workflow

The main CI workflow performs checkout, dependency installation, static checks, recursive test execution, and recursive build execution. A second workflow installs Playwright browsers and runs frontend end-to-end tests. These workflows show that the repository uses one dominant command surface even though the frontend still needs a dedicated E2E path.

IV. Preliminary Results

A. Configuration Surface and Test Inventory

Repository inspection found seven automated test files and two CI workflows. Of the seven test files, six belong to unit or component-style automation and one belongs to end-to-end testing. The backend contributes four spec files, the frontend contributes one unit file, one browser component file, and one Playwright file, and the shared package contributes one test file. This distribution shows that the current automated baseline is concentrated on helper logic and lightweight UI checks rather than on cross-package interaction paths.

TABLE IV
Observed testing inventory and configuration surface

Area	Unit / component	E2E	Config files	Main config locations
Frontend	2	1	5	vite.config.ts, vitest.config.ts, vitest.unit.config.ts, vitest.browser.config.ts, playwright.config.ts
Backend	4	0	1	vite.config.ts
Shared utils	1	0	1	vite.config.ts
CI workflows	N/A	N/A	2	.github/workflows/ci.yml, .github/workflows/e2e.yml

The frontend has the richest test-stack composition and the largest local configuration surface. The backend and shared package each depend on a single Vite+ configuration file. This asymmetry is relevant to the later failure analysis because the only observed inconsistency during direct execution appears in the frontend package.

B. Direct Execution Outcomes

Direct test execution produced stable results in the backend and shared package. vp run @circles/backend#test completed successfully with 4 passing files and 21 passing tests. vp run utils#test:coverage completed successfully with 1 passing file and 1 passing test. The frontend produced mixed results. The aggregated command vp run @circles/frontend#test reported one passing file, one failed suite, and an initialization failure in src/__tests__/utils.unit.spec.ts. However, the isolated unit command vp test run --config vitest.unit.config.ts passed with 1 passing file and 4 passing tests.

TABLE V
Observed execution outcomes on 2026-04-10

Command	Package / scope	Files	Tests	Result	Observed notes
vp run @circles/backend#test	Backend	4 passed	21 passed	Pass	Stable package-level execution
vp run @circles/frontend#test	Frontend aggregated	1 passed, 1 failed	2 passed	Fail	Initialization error in unit suite during multi-project run
vp test run --config vitest.unit.config.ts	Frontend unit config	1 passed	4 passed	Pass	Unit project succeeds in isolation
vp run utils#test:coverage	Shared utils	1 passed	1 passed	Pass	Stable single-package execution

The failing frontend run is important because it changes the interpretation of consistency. A shared CLI entry point is present, but consistent outcomes are not guaranteed when multiple frontend test projects are composed under one aggregate command. The error message, Cannot read properties of undefined (reading 'config'), appears before the unit assertions execute. This indicates an environment or runner initialization problem rather than a fault in the tested utility function itself.

C. Coverage and Detectability

Coverage commands show strong detectability for the currently targeted helper modules and very limited breadth beyond that scope. Backend coverage reached 100% statement, branch, function, and line coverage for range.ts, retry.ts, spotify-export.ts, and zip.ts. Frontend unit coverage reached 100% for src/lib/utils.ts. Shared-package coverage reached 100% for packages/utils/index.ts. These results confirm that the automated baseline is precise but narrow.

TABLE VI
Observed coverage for the currently instrumented scope

Package	Covered target	Statements	Branches	Functions	Lines
Backend	range.ts, retry.ts, spotify-export.ts, zip.ts	100%	100%	100%	100%
Frontend unit	src/lib/utils.ts	100%	100%	100%	100%
Shared utils	index.ts	100%	100%	100%	100%

The coverage data should not be interpreted as system-wide completeness. In the frontend, coverage is restricted to one utility module. In the backend, controller and workflow layers remain outside the reported coverage target set even though they are likely to carry higher integration risk. Therefore, detectability is high inside the instrumented helper scope and lower outside it.

D. Failure Pattern Analysis

Only one concrete failure was observed during direct execution, but it is analytically useful because it was not predicted by simple per-file unit reasoning. The failure affected the frontend unit suite during the aggregated frontend command and did not reproduce in the isolated unit-project run. This creates a difference between local project success and combined pipeline behavior.

TABLE VII
Observed failure evidence

Failure ID	Affected module	Failure type	Frequency	Interpretation
F01	src/__tests__/utils.unit.spec.ts	Runner or configuration initialization error	1 observed aggregated run	The command surface is unified, but the composed frontend test environment is still sensitive to project interaction

This result increases the apparent risk of frontend test orchestration and lowers confidence in detectability at the package boundary. The backend and shared package currently show no analogous evidence of instability. At this stage, the strongest preliminary finding is therefore mixed: unification simplifies the command surface and supports reproducible coverage reporting, but consistency claims remain weaker in the most configuration-heavy package.

V. Discussion

The preliminary results support a narrower claim than simple toolchain advocacy. Circles shows that unification improves reproducibility at the command and workflow level. Most test commands are routed through vp, coverage output is structurally similar across packages, and the CI workflows follow a compact execution pattern. At the same

time, the frontend failure demonstrates that consistency cannot be inferred from command unification alone. When the frontend unit and browser projects are composed under the aggregate frontend test command, an initialization failure appears that is absent in the isolated unit run. This indicates that environment interaction remains an active risk even inside a unified stack.

This finding is useful for QA analysis because it separates **surface-level unification** from **behavioral uniformity**. Surface-level unification means that packages share one visible command model, similar output conventions, and a common core toolchain. Behavioral uniformity is stronger. It requires that combined execution produce the same result as isolated execution for equivalent tests. The Circles data currently supports the first condition more strongly than the second.

A. Implications for Risk and Test Strategy

From a risk perspective, the backend helper layer currently has high detectability and low observed execution risk. It has four stable spec files, 21 passing tests, and 100% coverage within the configured target set. The shared package shows the same pattern on a smaller scale. The frontend should be interpreted differently. The current frontend helper scope is small and well-covered inside its target module, but the aggregate execution anomaly increases the likelihood that configuration interaction faults remain hidden until the full package command is run. In practical QA terms, this means that isolated success is not sufficient evidence for release confidence in the frontend package.

The results also show why narrow coverage percentages can be misleading. All observed coverage values are 100%, but they apply to a very small portion of the system. Controllers, route-level data loading, worker orchestration, and end-to-end user flows are still lightly represented in the measured baseline. The present evidence therefore supports a strategy that expands from helper-level certainty toward integration-level uncertainty, especially in the frontend and backend controller layers.

B. Threats to Validity

The study has four main threats to validity. First, it is a single-repository case study, so external validity is limited. Second, the evidence is based on the current repository state rather than on a historical before-and-after migration dataset. This means the paper can evaluate the present unified setup and its remaining inconsistencies, but it cannot quantify the exact delta from a prior fragmented implementation inside the same repository. Third, some coverage results are affected by explicit include lists in package configuration. These measurements are accurate for the configured target scope but not for the system as a whole. Fourth, the observed frontend failure was captured during direct execution on one evaluation date. It should therefore be treated as evidence of a real inconsistency, but not yet as a long-run flakiness estimate.

C. Generalizability and Limitations

The case still supports a general principle that is broader than Vite+. When multiple packages share one resolver model, one transformation stack, and one main command interface, the number of environment differences that can invalidate test comparisons is reduced. This principle should apply to other unified toolchains as well. However, the same case also shows a limitation of unification. Package-specific needs, such as browser projects or dedicated end-to-end configuration, reintroduce local variation. A unified toolchain can reduce fragmentation, but it does not eliminate the need to test cross-project composition explicitly.

VI. Conclusion

This paper examined testing consistency in the Circles TypeScript monorepo through repository inspection, CI workflow analysis, and direct execution of package test and coverage commands. The current repository demonstrates a clear unification pattern: most testing activity is routed through the vp command surface, the CI workflows are compact, and helper-level coverage reports are reproducible across packages. These properties reduce the visible configuration burden of running tests in a multi-package repository.

The empirical results are nonetheless mixed rather than absolute. Backend and shared-package execution were stable, and the targeted helper modules reported full coverage within their configured scope. The frontend package exposed a configuration-level initialization failure during its aggregated test command even though the same unit project passed in isolation. This shows that unified tooling improves consistency conditions, but consistency still has to be verified at the combined execution level. Future work should expand measurement toward route, controller, workflow, and end-to-end layers, and should collect repeated-run data to estimate flakiness and pipeline-level stability over time.

Acknowledgment

References

[1] J. Bogner and M. Merkel, "To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub," in *Proceedings of the 19th International Conference on Mining Software Repositories*, ACM, 2022, pp. 658–669. doi: 10.1145/3524842.3528454.

[2] T. Tang, S. Alimadadi, and N. Sumner, "From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem," in *Proceedings of the 23rd International Conference on Mining Software Repositories*, ACM, 2026, pp. 1–11. doi: 10.1145/3793302.3793379.

[3] Y. Wang, M. V. Mäntylä, Z. Liu, and J. Markkula, "Test Automation Maturity Improves Product Quality: Quantitative Study of Open Source Projects Using Continuous Integration," *Journal of Systems and Software*, vol. 188, p. 111259, 2022, doi: 10.1016/j.jss.2022.111259.

[4] L. Yu, E. Alégroth, P. Chatzipetrou, and T. Gorschek, "Automated NFR Testing in Continuous Integration Environments: A Multi-Case Study of Nordic Companies," *Empirical Software Engineering*, vol. 28, no. 144, 2023, doi: 10.1007/s10664-023-10356-1.